

Beethoven 项目数学基础理论

本文档系统总结 Beethoven 音乐多乐器声源分离项目涉及的数学理论，涵盖信号处理、线性代数、概率统计、深度学习、音频乐理等领域。

目录

- 信号处理数学基础
- 线性代数与矩阵分解
- 概率论与统计
- 深度学习数学
- 音频与乐理数学
- 优化算法
- 谱减与掩码数学
- 评价指标
- 总结：数学与项目的对应关系

1. 信号处理数学基础

1.1 傅里叶变换

傅里叶变换是整个音频信号处理的基石。它将时域信号分解为频率分量：

连续傅里叶变换 (CFT):

$$X(f) = \int_{-\infty}^{\infty} x(t) e^{-j2\pi ft} dt$$

逆变换:

$$x(t) = \int_{-\infty}^{\infty} X(f) e^{j2\pi ft} df$$

物理含义：任意连续信号 $x(t)$ 可以表示为不同频率正弦波的加权叠加。 $X(f)$ 表示频率 f 处的振幅和相位。

离散傅里叶变换 (DFT):

$$X[k] = \sum_{n=0}^{N-1} x[n] e^{-j\frac{2\pi}{N}kn} \quad k = 0, 1, \dots, N-1$$

其中 N 是采样点数， k 对应数字频率 $\omega_k = \frac{2\pi k}{N}$ 。

快速傅里叶变换 (FFT): DFT 的高效算法实现，时间复杂度 $O(N \log N)$ 而非 $O(N^2)$ 。

在项目中的角色： FFT 是所有频谱分析的核心工具，Demucs、U-Net、NMF 全部工作在频谱域而非时域。

1.2 短时傅里叶变换 (STFT)

音乐信号是非平稳的——频率成分随时间变化。STFT 通过在时间轴上加窗并逐段做 FFT，获得时间-频率二维表示：

$$X(m, k) = \sum_{n=-\infty}^{\infty} x[n] w[n-m] e^{-j\frac{2\pi}{N}kn}$$

其中：
- m ：帧索引（时间位置）
- k ：频率索引
- $w[n]$ ：窗函数（如 Hann 窗）

窗函数： 为了减少频谱泄漏，我们对每段加上平滑窗：

$$[w_{\text{Hann}}][n] = 0.5 \left[1 - \cos \left(\frac{2\pi}{L-1} n \right) \right], \quad 0 \leq n \leq L-1$$

项目参数：

参数	Phase 1-3	Web App	含义
n_fft	1024	2048	FFT 点数
hop_length	256	256	帧移（相邻窗重叠度）
窗类型	Hann	Hann	频谱泄漏控制
频率分辨率	$\frac{22050}{1024} \approx 21.5\text{Hz}$	$\frac{22050}{2048} \approx 10.8\text{Hz}$	每 bin 宽度

不确定性原理的体现：

$$[\Delta t \cdot \Delta f \geq \frac{1}{4\pi}]$$

时间分辨率和频率分辨率不可兼得。n_fft 越大，频率分辨率越高但时间分辨率越低；hop_length 越小，时间分辨率越高但计算量越大。

1.3 频谱幅度与相位

STFT 输出是复数矩阵，可分解为：

$$[X(m, k) = |X(m, k)| \cdot e^{j\angle X(m, k)}]$$

幅度谱：
$$|X(m, k)| = \sqrt{\text{Re}(X)^2 + \text{Im}(X)^2}$$

相位谱：
$$\angle X(m, k) = \arctan \left(\frac{\text{Im}(X)}{\text{Re}(X)} \right)$$

对数幅度谱（模型输入）：

$$[\text{log-spec}] = \log(1 + |X(m, k)|)$$

取对数的原因： 1. 人耳对声音的感知是对数级的（韦伯-费希纳定律） 2. 压缩动态范围，使弱信号也可见 3. 更符合神经网络对输入分布的要求

在项目中的角色： 神经网络处理幅度谱（对数域），生成掩码后结合原始相位做 ISTFT 重建音频。

1.4 逆短时傅里叶变换 (ISTFT)

将 STFT 复数矩阵重建为时域信号：

$$[x[n] = \frac{\sum_m y_m[n] w[n-m]}{\sum_m w^2[n-m]}]$$

其中 y_m[n] 是第 m 帧的逆 FFT 结果。分母是**重叠相加 (Overlap-Add)** 归一化因子。

相位问题： 在源分离中，我们修改幅度谱后需要结合原始混合信号的相位做 ISTFT。这假设了相位信息在以人耳感知的尺度上足够接近原始信号 —— 这是合理的近似，因为人耳对相位不敏感。

1.5 采样定理 (Nyquist)

奈奎斯特-香农采样定理：

$$[f_s \geq 2f_{\text{max}}]$$

其中 f_s 是采样率，f_{max} 是信号最大频率。

项目设置: $f_s = 22050\text{Hz}$, 即可表示 $0 \sim 11025\text{Hz}$ 的频率范围, 覆盖音乐信号的主要频段 (钢琴最高音约 4186Hz, 泛音可达 10kHz+)。

2. 线性代数与矩阵分解

2.1 非负矩阵分解 (NMF)

NMF 是 Phase 1 基线的核心技术。它将非负矩阵 V 分解为两个非负矩阵的乘积:

$$[V_{\{F \times T\}} \approx W_{\{F \times K\}} \cdot H_{\{K \times T\}}]$$

其中: - V : 幅度谱 (F 个频率 bin $\times$ T 个时间帧) - W : 基矩阵 (K 个频谱模板) - H : 激活矩阵 (每个模板随时间的变化) - K : 分解秩 (谱模板数量)

约束条件:

$$[W \geq 0, \quad H \geq 0]$$

优化目标 (Frobenius 范数):

$$[\min_{\{W, H \geq 0\}} \|V - WH\|_F^2 = \min_{\{W, H \geq 0\}} \sum_{i,j} (V_{ij} - (WH)_{ij})^2]$$

乘法更新规则 (Lee & Seung, 2000):

$$[H \leftarrow H \odot \frac{W^T V}{W^T W H + \epsilon}]$$

$$[W \leftarrow W \odot \frac{V H^T}{W H H^T + \epsilon}]$$

其中 $\odot$ 是逐元素乘法, 除法是逐元素除法。 ϵ 是防除零的小常数。

用于声源分离的流程:

- 训练阶段: 对每个乐器单独训练 W_i (固定基模板)
- 分离阶段: 固定 $W = [W_1, W_2, \dots, W_N]$, 仅更新 H
- 重建: 第 i 个乐器的频谱 = $W_i H_i$

项目结果: NMF 在合成数据上达到 SDR 0.6~1.8 dB, 作为基线验证了问题的可解性。后续深度学习方法的 SDR 达到 11~17 dB。

2.2 矩阵范数与距离

Frobenius 范数:

$$[\|A\|_F = \sqrt{\sum_{i=1}^m \sum_{j=1}^n |a_{ij}|^2}]$$

等价于向量化的 l_2 范数。

L1 范数 (用于损失函数):

$$[\|A\|_1 = \sum_{i,j} |a_{ij}|]$$

2.3 卷积运算

一维卷积:

$$[(x * w)[n] = \sum_{k=-\infty}^{\infty} x[k] w[n-k]]$$

二维卷积 (CNN 核心) :

$$[(I * K)[i, j] = \sum_m \sum_n I[m, n] \cdot K[i-m, j-n]]$$

实际实现中通常使用互相关 (数学上等价于卷积的旋转版本) :

$$[(I \star K)[i, j] = \sum_m \sum_n I[i+m, j+n] \cdot K[m, n]]$$

转置卷积 (反卷积) :

用于 U-Net 解码器的上采样, 将低分辨率特征图映射回高分辨率。

$$[y = W^T x + b]$$

其中 W 是卷积矩阵, W^T 是其转置。

在项目中的角色: Conv2d 是 U-Net 和 U-Net++ 的基本构建块, 每个残差块包含 2 个 Conv2d 层。

2.4 张量运算

PyTorch 中的张量运算遵循**广播 (Broadcasting)** 规则:

$$[\text{shape}(A) = (m, n), \quad \text{shape}(B) = (n)]$$

$$[A + B \rightarrow \text{shape}(A) = (m, n), \quad \text{shape}(B) = (1, n) \rightarrow \text{broadcast} (m, n)]$$

在模型中, 频谱张量的形状流转为:

$$[\text{Batch} \times 1 \times \text{Freq} \times \text{Time} \rightarrow \text{encoder} \dots \rightarrow \text{decoder} \times \text{Batch} \times C \times \text{Freq} \times \text{Time}]$$

最后用 1×1 卷积输出 C 个乐器的掩码。

3. 概率论与统计

3.1 基本统计量

均方根 (RMS): 表征信号能量

$$[\text{RMS}] = \sqrt{\frac{1}{N} \sum_{i=1}^N x_i^2}$$

均值:
$$\bar{x} = \frac{1}{N} \sum_{i=1}^N x_i$$

方差:
$$\sigma^2 = \frac{1}{N} \sum_{i=1}^N (x_i - \bar{x})^2$$

在项目中的角色: RMS 用于评估分离后各音轨的音量水平, 在损失函数中比较预测和目标之间的能量差异。

3.2 Pearson 相关系数

衡量两个信号之间的线性相关程度:

$$[\rho_{xy} = \frac{\sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum_{i=1}^n (x_i - \bar{x})^2} \sqrt{\sum_{i=1}^n (y_i - \bar{y})^2}}]$$

取值范围: $\rho \in [-1, 1]$

- $\rho = 1$: 完全正相关 (信号形状相同)

- $\rho = 0$: 不相关
- $\rho = -1$: 完全负相关 (反相)

在项目中的角色：在构想二 (Approach 2) 的置信度滤波器中，用滑动窗口的相关系数判断合成信号与原始混合/分离轨的匹配程度。 $\rho \cdot \min(\frac{r_t}{r_o}, \frac{r_o}{r_t}) > 0.15$ 时保留该片段。

3.3 损失函数

L1 损失 (平均绝对误差)：

$$\mathcal{L}_1 = \frac{1}{N} \sum_{i=1}^N |\hat{y}_i - y_i|$$

L2 损失 (均方误差)：

$$\mathcal{L}_2 = \frac{1}{N} \sum_{i=1}^N (\hat{y}_i - y_i)^2$$

项目使用的组合损失：

$$\mathcal{L} = \mathcal{L}_1 + \lambda \mathcal{L}_{TC}$$

其中 $\mathcal{L}_{TC}$ 是**时序连续性损失** (Temporal Continuity Loss)：

$$\mathcal{L}_{TC} = \frac{1}{C \cdot F \cdot (T-1)} \sum_{c=1}^C \sum_{f=1}^F \sum_{t=1}^{T-1} (M_{c,f,t+1} - M_{c,f,t})^2$$

这约束了掩码在时间轴上平滑变化 —— 对应原始构想中的"时序连续性约束"，惩罚掩码的帧间突变。

在 Phase 3 中， $\lambda = 0.05$ 。

3.4 数据归一化

Batch Normalization：

$$\hat{x}_i = \frac{x_i - \mu}{\sqrt{\sigma^2 + \epsilon}}$$

$$y_i = \gamma \hat{x}_i + \beta$$

其中 μ_B 和 σ_B^2 是当前 mini-batch 的均值和方差， γ 和 β 是可学习参数。

作用： 1. 缓解内部协变量偏移 (Internal Covariate Shift) 2. 允许更大的学习率 3. 具有轻微的正则化效果

在项目中的角色：U-Net 和 U-Net++ 的每个卷积后都接 BatchNorm2d。

4. 深度学习数学

4.1 神经网络基础运算

线性 (全连接) 层：

$$y = Wx + b$$

其中 $W \in \mathbb{R}^{m \times n}$ 是权重矩阵， $b \in \mathbb{R}^m$ 是偏置向量。

修正线性单元 (ReLU)：

$$\text{ReLU}(x) = \max(0, x)$$

Sigmoid 函数：

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

输出范围 $(0, 1)$ ，用于生成掩码（因为掩码需要介于 0 和 1 之间）。

链式法则（反向传播基础）：

$$\frac{\partial \mathcal{L}}{\partial x} = \frac{\partial \mathcal{L}}{\partial y} \cdot \frac{\partial y}{\partial x}$$
$$\frac{\partial \mathcal{L}}{\partial W} = \frac{\partial \mathcal{L}}{\partial y} \cdot x^T$$

4.2 U-Net 架构数学

U-Net 是一个编码器-解码器结构，核心数学操作：

编码器（下采样）：

$$x_{\text{enc}}^{(l)} = \text{Pool}(\text{Conv}(x_{\text{enc}}^{(l-1)}))$$

每次下采样：频率维度 × 时间维度各减半，通道数加倍。

解码器（上采样）：

$$x_{\text{dec}}^{(l)} = \text{Conv}(\text{Concat}(x_{\text{enc}}^{(L-l)}, \text{UpConv}(x_{\text{dec}}^{(l-1)})))$$

跳跃连接 (Skip Connection)：

编码器第 l 层的特征图直接拼接到解码器对应层的输入。这确保了高分辨率的局部细节不会在编解码过程中丢失。

项目 U-Net 通道流转：

输入：1 ch	
Enc1: 1 → 16 (Phase 2) / 24 (Phase 3)	特征图：F × T
Enc2: 16 → 32 / 24 → 48	特征图：F/2 × T/2
Enc3: 32 → 64 / 48 → 96	特征图：F/4 × T/4
Enc4: 64 → 128 / 96 → 192	特征图：F/8 × T/8
Bot: 128 → 256 / 192 → 384	特征图：F/16 × T/16
Dec4: 384 → 192	跳跃连接 Enc4
Dec3: 192 → 96	跳跃连接 Enc3
Dec2: 96 → 48	跳跃连接 Enc2
Dec1: 48 → 24	跳跃连接 Enc1
输出：24 → C (3 或 4 个乐器)	

参数量计算：

对于卷积层： $\text{params} = C_{\text{in}} \times C_{\text{out}} \times K_h \times K_w + C_{\text{out}}$ （偏置）

$K_h \times K_w = 3 \times 3$ 是我们使用的卷积核大小。

Phase 2 U-Net 总参数量：约 **194 万** Phase 3 U-Net++ 总参数量：约 **658 万**

4.3 ResBlock (残差块)

ResBlock 通过引入"捷径连接"解决深层网络梯度消失问题：

$$\mathbf{y} = \mathcal{F}(\mathbf{x}) + \mathbf{x}$$

其中 $\mathcal{F}(\mathbf{x})$ 是残差映射：

$$\mathcal{F}(\mathbf{x}) = \text{Conv}(\text{BN}(\text{ReLU}(\text{Conv}(\text{BN}(\mathbf{x}))))$$

梯度传播分析：

$$\frac{\partial \mathcal{L}}{\partial \mathbf{x}} = \frac{\partial \mathcal{L}}{\partial \mathbf{y}} \cdot \left(1 + \frac{\partial \mathcal{F}}{\partial \mathbf{x}}\right)$$

关键点：梯度通过 $\frac{\partial \mathcal{L}}{\partial \mathbf{y}} \cdot 1$ 这条"高速公路"直达浅层，即使 $\frac{\partial \mathcal{F}}{\partial \mathbf{x}}$ 很小（梯度消失），整体梯度也不会消失。

4.4 掩码机制 (Masking)

这是整个项目的核心技术假设：

$$[S_i = M_i \odot X_{\text{mix}}]$$

其中： - X_{mix} ：混合信号的频谱（幅度） - M_i ：第 i 个乐器的掩码， $M_i \in [0, 1]$ - S_i ：第 i 个乐器的分离频谱
- $\odot$ ：逐元素乘法

理想掩码 (Ideal Mask) 理论：

假设源信号 S_i 已知，理想掩码为：

$$[M_i^{\text{ideal}} = \frac{[S_i]}{[X_{\text{mix}}]}]$$

通常满足 $\sum_i M_i^{\text{ideal}} \leq 1$ （能量守恒）。

PSM (Phase-Sensitive Mask)：

$$[M_i^{\text{PSM}} = \frac{[S_i]}{[X_{\text{mix}}]} \cos(\theta_{\text{mix}} - \theta_i)]$$

考虑了相位差异，更准确。

项目实施：

模型输出 $\hat{M} = \sigma(\text{Conv}_{1 \times 1}(\text{features}))$ ，Sigmoid 函数保证输出在 $[0, 1]$ 。

4.5 教师-学生蒸馏 (Teacher-Student Distillation)

核心思想： 用大模型（教师）的输出作为小模型（学生）的训练目标。

$$[\text{distill}] = |f_{\text{student}}(x) - f_{\text{teacher}}(x)|$$

为什么有效：

- 教师模型的输出包含“暗知识”——不仅告诉学生正确的分类，还告诉它哪些类别相似
- 在源分离中，教师输出的是连续的掩码值（而非离散标签），包含丰富的结构信息

项目实施 (Phase 3)：

- 教师：Demucs (Hybrid Transformer Demucs)
- 学生：U-Net++ (658 万参数)
- 数据：真实歌曲（BEYOND - 光辉岁月）
- 结果：经过 17 段 6 秒切片、30 轮训练，学生达到教师 66-98% 的水平

4.6 神经网络训练的数学流程

- 前向传播：** $\hat{y} = f_{\theta}(x)$
- 损失计算：** $\mathcal{L} = \ell(\hat{y}, y)$
- 反向传播：** $\nabla_{\theta} \mathcal{L} = \frac{\partial \mathcal{L}}{\partial \theta}$
- 参数更新 (Adam)：** $[m_t = \beta_1 m_{t-1} + (1-\beta_1) \nabla_{\theta} \mathcal{L}]$ $[v_t = \beta_2 v_{t-1} + (1-\beta_2) (\nabla_{\theta} \mathcal{L})^2]$ $[\hat{m}_t = \frac{m_t}{1-\beta_1^t}, \hat{v}_t = \frac{v_t}{1-\beta_2^t}]$ $[\theta_{t+1} = \theta_t - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}]$

5. 音频与乐理数学

5.1 音高与频率

十二平均律： 将一个八度均分为 12 个半音，相邻半音频率比为 $2^{1/12}$ 。

频率转 MIDI 音符：

$$[n = 69 + 12 \cdot \log_2(\frac{f}{440})]$$

其中 $n = 69$ 对应 A4 (440Hz)。

MIDI 转频率：

$$[f = 440 \cdot 2^{\frac{n-69}{12}}]$$

音符命名与频率对应：

MIDI 音符	名称	频率
69	A4	440.00 Hz
60	C4 (中央C)	261.63 Hz
72	C5	523.25 Hz
81	A5	880.00 Hz

在项目中的角色： 构想二将检测到的音高 (Hz) 转换为 MIDI 音符号，用于 MIDI 文件生成和音符合并判断。

5.2 谐波级数 (Harmonic Series)

乐器音色由其谐波结构决定。一个周期波的谐波：

$$[f_n = n \cdot f_0, \text{quad } n = 1, 2, 3, \dots]$$

- $n = 1$: 基频 (fundamental)
- $n = 2, 3, \dots$: 泛音 (overtones)

项目合成器 (Approach 2) 的谐波参数：

钢琴：谐波 [1,2,3,4,5,6,8] 权重 [1, .6, .3, .15, .08, .04, .02]
贝斯：谐波 [1,3] 权重 [1, .2]
吉他：谐波 [1,2,3,4,5] 权重 [1, .4, .2, .1, .05]
人声：谐波 [1,2,3,4,5] 权重 [1, .3, .1, .05, .02] + 颤音
打击乐：谐波 [1,2,3] 权重 [1, .5, .25] + 噪声

合成公式：

$$[x_{\text{note}}(t) = \sum_{h=1}^H w_h \cdot \sin(2\pi \cdot h \cdot f_0 \cdot t) \cdot e(t)]$$

其中 $e(t)$ 是包络函数, w_h 是各谐波的权重。

5.3 包络 (Envelope)

指数衰减：

$$[e_{\text{decay}}(t) = e^{-t \cdot \tau}]$$

其中 τ 是衰减速率（不同乐器不同，钢琴快、管乐慢）。

起音 (Attack):

$$[a(t) = \min\left(1, \frac{t}{t_{\text{attack}}}\right)]$$

完整包络:

$$[e(t) = a(t) \cdot e^{-\gamma t}]$$

5.4 颤音 (Vibrato)

低频频率调制，使人声和弦乐器音色更自然：

$$[f(t) = f_0 \cdot (1 + \alpha \cdot \sin(2\pi f_{\text{vib}} t))]$$

相位:
$$\phi(t) = 2\pi \int_0^t f(\tau) d\tau$$

项目参数: $\alpha = 0.05, f_{\text{vib}} = 5.5\text{Hz}$

5.5 峰值检测 (Spectral Peak Picking)

音高检测的核心是寻找频谱中的局部极大值：

数学条件:

$$[\frac{dX}{df} = 0, \quad \frac{d^2X}{df^2} < 0]$$

离散实现 (`scipy.signal.find_peaks`) :

$$[X[k-1] < X[k] > X[k+1]]$$

突出度 (Prominence): 峰值相对于周围最低鞍点的高度。用来筛选有意义的峰值，忽略噪声引起的毛刺。

阈值条件:

$$[X_{\text{peak}} > \max(X) \cdot \theta, \quad \theta = 0.05 \sim 0.1]$$

5.6 音符合并 (Note Grouping)

将分散的音高检测结果合并为连续音符：

频率相近性检查:

$$[\frac{|f_1 - f_2|}{f_1} < \theta_f, \quad \theta_f = 0.05 \sim 0.06]$$

时间连续性检查:

$$[t_{\text{end},1} - t_{\text{start},2} < \Delta t, \quad \Delta t \approx 50-80\text{ms}]$$

当两个检测同时满足频率相近和时间连续条件时，合并为同一个音符。

5.7 音量到 MIDI 速度 (Velocity) 映射

$$[\text{velocity} = \min(127, \text{int}(\text{conf} \times 80 + 30))]$$

映射到 MIDI 标准的 0-127 范围。

6. 优化算法

6.1 Adam 优化器

Adam (Adaptive Moment Estimation) 是项目中所有神经网络训练使用的优化器。

结合了 Momentum (动量法) 和 RMSProp 的优点：

1. **动量项**（一阶矩估计）： $m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t$ - 记录梯度方向的历史，类似物理中的惯性
2. **自适应学习率**（二阶矩估计）： $v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$ - 每个参数独立的学习率，梯度大的维度步长小
3. **偏差校正**（初始时刻 $m_0, v_0 = 0$ 的修正）：

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1 - \beta_2^t}$$

4. **参数更新**：

$$\theta_{t+1} = \theta_t - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}$$

项目超参数： - 学习率 $\eta = 5 \times 10^{-4}$ - $\beta_1 = 0.9$ （默认） - $\beta_2 = 0.999$ （默认） - $\epsilon = 10^{-8}$ （默认）

6.2 Mini-Batch 梯度下降

使用小批量（batch）数据计算梯度：

$$g_t = \frac{1}{|\mathcal{B}|} \sum_{i \in \mathcal{B}} \nabla_{\theta} \ell(x_i, y_i; \theta)$$

项目设置：batch_size = 4（受 CPU 内存限制）

7. 谱减与掩码数学

7.1 幅度掩码的数学特性

假设混合信号 X_{mix} 由 N 个源组成：

$$X_{\text{mix}} = S_1 + S_2 + \dots + S_N$$

在幅度谱域，取绝对值后不再满足线性叠加：

$$|X_{\text{mix}}| \neq |S_1| + |S_2| + \dots + |S_N|$$

这是因为存在相位干涉。但如果相位均匀分布，统计上：

$$\mathbb{E}[|X_{\text{mix}}|^2] \approx \mathbb{E}[|S_1|^2] + \mathbb{E}[|S_2|^2] + \dots + \mathbb{E}[|S_N|^2]$$

这就是**功率谱的加性假设**——源分离在功率谱域比幅度谱域更合理。

7.2 软掩码 vs 硬掩码

硬掩码（二值掩码）：

$$M_i[f, t] = \begin{cases} 1 & \text{如果乐器 } i \text{ 在 } (f, t) \text{ 处占主导} \\ 0 & \text{否则} \end{cases}$$

软掩码（连续值掩码）：

$$M_i[f, t] \in [0, 1]$$

项目使用软掩码，让模型输出连续值。

7.3 Overlap-Add 推理

在 Phase 3 推理中，由于内存限制，将长音频切片处理，结果叠加平均：

$$M_{\text{final}}[f, t] = \frac{\sum_c M_c[f, t] \cdot \mathbb{I}(t \in \text{chunk}_c)}{\sum_c \mathbb{I}(t \in \text{chunk}_c)}$$

每段重叠 50%，在重叠区域取平均，避免切块边界处的跳变。

8. 评价指标

8.1 SDR (Signal-to-Distortion Ratio)

SDR 是声源分离最主要的客观评价指标：

$$[\text{SDR}] = 10 \cdot \log_{10} \frac{s_{\text{target}}^2}{e_{\text{distortion}}^2}$$

其中 $e_{\text{distortion}}$ 包含： - 来自其他源的交渗 (interference) - 人工噪声 (noise) - 算法失真 (artifact)

SDR 的物理含义：

SDR 值	感知质量
> 15 dB	极好，几乎无失真
10-15 dB	好，轻微失真
5-10 dB	可接受，明显失真
< 5 dB	差，分离效果不佳

8.2 项目评测方法

由于没有标准数据集（MUSDB18 在中国无法下载），项目使用相对度量：

$$[\text{接近程度}] = \frac{\text{学生 RMS}}{\text{教师 RMS}} \times 100\%$$

虽然 RMS 本身不能完全反映感知质量（RMS 相等不代表音质相同），但它是一个直观的代理指标。

8.3 相关性与体积比联合筛选

构想二使用：

$$[\text{score}] = \rho_{xy} \cdot \min\left(\frac{r_t}{r_o}, \frac{r_o}{r_t}\right) \geq [\text{threshold}]$$

其中 ρ_{xy} 是合成信号与原始信号滑动窗口的相关系数， $\min(r_t/r_o, r_o/r_t)$ 是音量比惩罚项，防止信号过小或过大。

9. 总结：数学与项目的对应关系

数学领域	具体理论	在项目中的位置
信号处理	STFT / ISTFT	数据预处理管线：所有音频都经过 STFT 到频谱域，分离后再 ISTFT 重建音频
信号处理	窗函数与重叠相加	频谱平滑与重建：Hann 窗减少频谱泄漏，OLA 保证重建连续性
信号处理	对数压缩	模型输入： $\log(1 + X)$ 作为 U-Net/U-Net++ 的输入特征
线性代数	NMF 分解	Phase 1 基线： $V \approx WH$ 乘法更新规则进行频谱模板分解
线性代数	卷积运算	U-Net 核心：Conv2d + 转置 Conv2d 实现编解码器
线性代数	张量运算	所有 PyTorch 操作：批量并行计算
概率统计	相关系数	构想二置信度滤波：判断合成信号与原始信号的匹配度
概率统计	L1/L2 损失	训练目标函数： $ \text{sep} - \text{target} _1 + \lambda \Delta \text{mask} _2^2$
概率统计	Batch Normalization	模型每层之后： $\gamma \frac{x - \mu}{\sigma} + \beta$

数学领域	具体理论	在项目中的位置
深度学习	U-Net 架构	Phase 2: 4 级编解码器 + 跳跃连接, 194 万参数
深度学习	ResBlock	Phase 3 关键升级: $y = F(x) + x$ 缓解梯度消失
深度学习	Sigmoid 输出	掩码生成: $\sigma(x) \in [0, 1]$ 作为软掩码
深度学习	教师-学生蒸馏	Phase 3 训练策略: U-Net++ 学习模仿 Demucs
音频乐理	十二平均律	构想二 MIDI 导出: $f = 440 \cdot 2^{(n - 69)/12}$
音频乐理	谐波级数	构想二合成器: 不同乐器有不同的谐波权重分布
音频乐理	包络 (ADSR)	构想二合成器: 指数衰减 + 起音, 模拟真实乐器音色
音频乐理	峰值检测	构想二音高检测: 频谱局部极大值 + 突出度筛选
优化算法	Adam	所有神经网络训练: 自适应学习率 + 动量
优化算法	Mini-batch GD	训练: batch_size=4, 受 CPU 内存限制

结语

Beethoven 项目从大一学生洗澡时的一个想法出发, 经历了信号处理基线 (NMF) → 深度学习 (U-Net) → 教师-学生训练 (U-Net++ 蒸馏) → 生成式分离 (构想二) 的完整探索。

整个项目的理论基础可以概括为三句话: 1. **物理上**, 混合音频 = 各乐器声波在空气中的线性叠加 (奈奎斯特采样 + STFT 分析) 2. **数学上**, 源分离 = 在频谱域求解 $X = \sum M_i \odot X$ 的 ill-posed 反问题 (时序连续性约束 + 深度学习先验 + 掩码机制) 3. **工程上**, 从 NMF → U-Net → 教师-学生蒸馏 → 生成式重演, 每一步都是对"如何拆解混合音频"这个问题的不同数学视角

每一种方法本质上都在做同一件事: 为"无数组解"的问题加上正确的约束条件, 筛选出那个符合真实演奏规律的解。